Engenharia de Sistemas Distribuídos

Grupo: José Samuel Alves da Silva, Fernando D'ávila Lins Bezerra Cavalcati Filho, João Vitor Cardoso Beltrão, Gabriel Dantas de Sousa

Sistema para controle e Solicitação de Serviços para o Setor de TI da Prefeitura Municipal de Jacaraú

Os itens obrigatórios desta entrega são:

1. **Definição do Stack Tecnológico e Justificativas**.
2. **Identificação dos padrões arquiteturais** (mínimo de 3).
3. **ADRs (Architecture Decision Records):** Pelo menos 2 decisões arquiteturais documentadas.
4. **Diagramas C4 (Níveis 1 e 2):** Contexto do Sistema e Containers (Microsserviços, API Gateway, Bancos de Dados).
5. **Setup inicial do ambiente** (Docker Compose, CI básico).
6. **Divisão de responsabilidades entre os integrantes** (equipe de 5 pessoas).

---

06 de Novembro de 2026

1. Stack Tecnológica em Python para o seu Projeto

- **Framework para as APIs (Microsserviços):**
    - **FastAPI:** É a melhor escolha em Python hoje para microsserviços. Ele é extremamente rápido, assíncrono por padrão (ótimo para lidar com o painel de chamados e conexões simultâneas), gera documentação automática via Swagger/OpenAPI e tem validação de dados nativa com *Pydantic*.
- **API Gateway:**
    - **FastAPI** para criar um microsserviço leve de Gateway que faz o roteamento das requisições e a validação de tokens JWT, ou utilizar ferramentas prontas como o *Kong*.
- **Mensageria e Processamento Assíncrono:**
    - **Celery** ou **RabbitMQ/Kafka com Pika/aiokafka:** Perfeito para o padrão de mensageria assíncrona. Quando um chamado é aberto, a API joga um evento na fila e um worker em Python consome esse evento para disparar o e-mail ou notificação sem travar a resposta do usuário.
- **Banco de Dados e ORM:**
    - **SQLAlchemy** (ou *Tortoise-ORM* para abordagens totalmente assíncronas) junto com **PostgreSQL** para garantir a consistência dos chamados e do inventário de equipamentos.

---

2. Padrões Arquiteturais Selecionados (Mínimo de 3 exigidos

)

1. **API Gateway Pattern:** Centraliza o ponto de entrada, gerencia autenticação (JWT) e protege o painel público contra tráfego abusivo (anti-throttling/rate-limit).
2. **Transactional Outbox Pattern / Event-Driven Architecture (EDA):** Garante a consistência ao criar um chamado e disparar a notificação por e-mail/WhatsApp de forma assíncrona via message broker, evitando falhas de comunicação síncrona.
3. **Database-per-Service:** Cada microsserviço possui sua própria base de dados isolada, garantindo o desacoplamento estrito exigido em arquiteturas distribuídas.

---

3. Arquitetura Decision Records (ADRs)

Aqui estão as duas primeiras ADRs documentadas no formato padrão exigido em arquitetura de software:

ADR 01: Adoção de Arquitetura de Microsserviços

- **Status:** Aceito
- **Contexto:** O sistema da prefeitura precisa atender a dois domínios principais e independentes: a gestão de chamados de TI (abertura pública e atendimento por técnicos) e o gerenciamento de inventário de equipamentos públicos (próprios e alugados, distribuídos por setores).
- **Decisão:** Adotar uma arquitetura baseada em **microsserviços** em vez de um monólito tradicional. O sistema será dividido em serviços autônomos utilizando **Python (FastAPI)**: um serviço dedicado a chamados (ticket-service) e outro dedicado ao inventário (asset-service), mediados por um API Gateway.
- **Consequências (Prós e Contras):**
    - *Prós:* Isolamento de falhas (se o serviço de inventário cair, a abertura de chamados continua funcionando), facilidade de manutenção e escalabilidade independente.
    - *Contras:* Maior complexidade operacional, necessidade de gerenciamento de rede e consistência eventual entre os serviços.

ADR 02: Adoção do Padrão Database-per-Service (Banco de Dados por Serviço)

- **Status:** Aceito
- **Contexto:** Cada microsserviço precisa gerenciar seus próprios dados sem acoplamento direto no nível de banco de dados para garantir a independência de deploy e evolução.
- **Decisão:** Cada microsserviço (ticket-service e asset-service) terá a sua própria instância isolada de banco de dados relacional (**PostgreSQL**). Nenhuma comunicação direta entre tabelas de serviços diferentes será permitida; qualquer compartilhamento de dados deve ocorrer via API ou mensageria.
- **Consequências (Prós e Contras):**
    - *Prós:* Desacoplamento estrito, autonomia de schema para cada equipe/serviço e prevenção de dependências cruzadas.
    - *Contras:* Desafios para manter consistência de dados em operações que envolvam múltiplos domínios (ex: vincular um chamado a um equipamento específico).

---

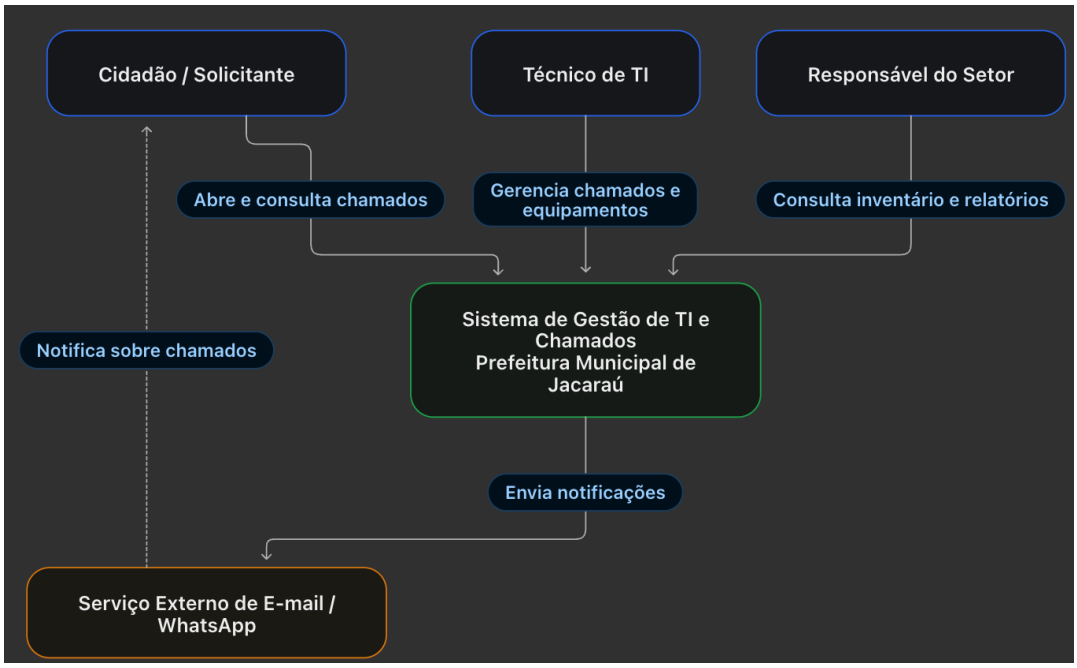
4. Diagrama C4 (Níveis 1 e 2) para o README

Esta estrutura em texto serve para você documentar visualmente e textualmente a arquitetura no README.md do repositório:

Nível 1: System Context Diagram (Diagrama de Contexto)

Mostra os usuários interagindo com o sistema global da prefeitura.

- **Usuários / Atores:**
    - **Cidadão / Solicitante:** Abre chamados de suporte técnico via painel público (sem login, informando e-mail, nome, WhatsApp e dados do problema com mídias).
    - **Técnico de TI (Você e seu colega):** Recebem, gerenciam, priorizam e respondem os chamados, além de administrar o inventário de equipamentos.
    - **Responsável do Setor:** Consulta os equipamentos alocados em seu respectivo setor e emite relatórios/impressões.
- **Sistema Principal:**
    - **Sistema de Gestão de TI e Chamados da Prefeitura:** Plataforma distribuída responsável por centralizar o atendimento via tickets e o controle patrimonial de hardware.

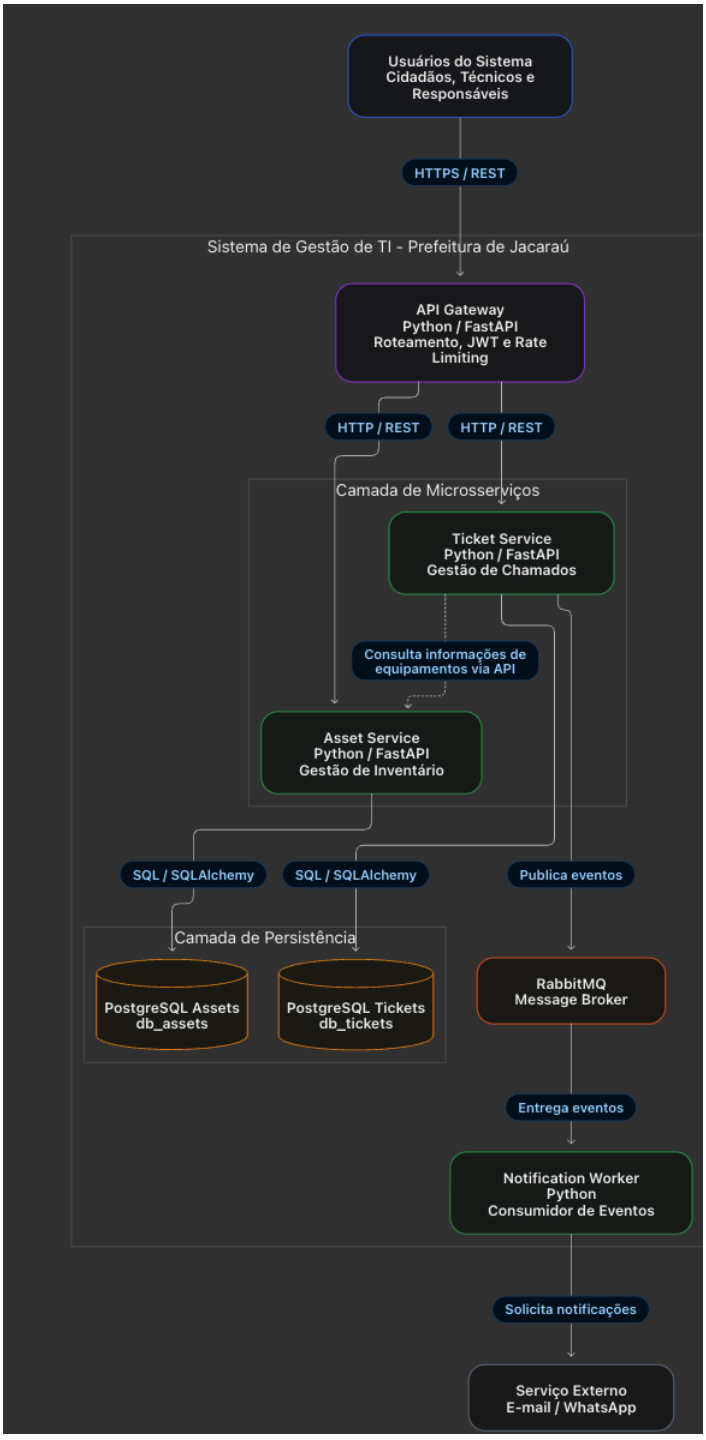


Nível 2: Containers Diagram (Diagrama de Containers)

Detalha os blocos executáveis (microsserviços e bancos de dados) que compõem a solução:

- **API Gateway (api-gateway):** Ponto único de entrada construído em Python. Gerencia o roteamento de requisições, segurança e rate-limiting para proteger o painel público.

- **Microsserviço de Chamados (ticket-service):** Desenvolvido em **FastAPI**. Gerencia a API de abertura de chamados, anexos de mídias e painel dos técnicos.
- **Microsserviço de Inventário (asset-service):** Desenvolvido em **FastAPI**. Controla o cadastro de equipamentos (próprios/alugados), histórico de movimentações entre setores e geração de relatórios.
- **Mensageria (rabbitmq):** Broker de mensagens para processamento assíncrono (ex: disparo de e-mails de notificação de novos chamados).
- **Bancos de Dados (postgres-tickets & postgres-assets):** Instâncias de PostgreSQL isoladas para cada microsserviço.



---

5. Arquivo Docker Compose Inicial (docker-compose.yml)

docker-compose.yml

```yaml
version: '3.8'

services:
  # Banco de dados do Microsserviço de Chamados
  postgres-tickets:
    image: postgres:15-alpine
    container_name: postgres_tickets
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secretpassword
      POSTGRES_DB: db_tickets
    ports:
      - "5432:5432"
    volumes:
      - pgdata-tickets:/var/lib/postgresql/data
    networks:
      - tsi-network

  # Banco de dados do Microsserviço de Inventário
  postgres-assets:
    image: postgres:15-alpine
    container_name: postgres_assets
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secretpassword
      POSTGRES_DB: db_assets
    ports:
      - "5433:5432"
    volumes:
      - pgdata-assets:/var/lib/postgresql/data
    networks:
      - tsi-network

  # Message Broker para Comunicação Assíncrona
  rabbitmq:
    image: rabbitmq:3-management-alpine
    container_name: rabbitmq_broker
    ports:
      - "5672:5672"
      - "15672:15672" # Painel de Gestão do RabbitMQ
    networks:
      - tsi-network

volumes:
  pgdata-tickets:
  pgdata-assets:

networks:
  tsi-network:
    driver: bridge
```

---

6. Sugestão Divisão de Responsabilidades da Equipe (5 Integrantes)

Constará no README do Projeto 02 e na documentação inicial:

- **Integrante 1 (Arquitetura e Infraestrutura / DevOps):**
    - Responsável pelo setup do ambiente com Docker Compose, configuração do RabbitMQ (broker de mensagens) e estruturação inicial do repositório Git com o pipeline de CI básico.
    - Configuração do API Gateway para roteamento de requisições e rate-limiting inicial.
- **Integrante 2 (Microsserviço de Chamados - Core & Banco de Dados):**
    - Responsável pelo desenvolvimento do ticket-service em Python (FastAPI).
    - Implementação da API de abertura de chamados públicos (sem login: e-mail, nome, WhatsApp, mídias) e persistência no banco relacional PostgreSQL.
- **Integrante 3 (Painel dos Técnicos e Notificações Assíncronas):**
    - Responsável pelo painel de atendimento dos técnicos (você e seu colega), controle de status e fila de atendimento.
    - Implementação do consumidor de eventos via mensageria para o disparo assíncrono de notificações por e-mail/WhatsApp (usando padrão Pub-Sub).
- **Integrante 4 (Microsserviço de Inventário e Gestão Patrimonial):**
    - Responsável pelo desenvolvimento do asset-service em Python (FastAPI).
    - Modelagem dos dados para gerenciar equipamentos próprios e alocados por setor, substituindo a planilha Excel atual.
- **Integrante 5 (Relatórios, Testes e Documentação):**
    - Responsável pela implementação da funcionalidade de consulta por setor e geração/impressão de termos/relatórios (PDFs).
    - Condução do plano de testes de integração, escrita final das ADRs e consolidação da documentação no README.